

Spring MVC lecture 2 ==>

In SpringMVC/SpringBoot-MVC the FrontController is "DispatcherServlet" which is ready made ServletComponent.

All the activities in SpringMVC takes place under the control or monitoring of DispatcherServlet Component.

Approches to develop Web app with Spring MVC Module:

=====

1. Declartive Cfgs Approach(XML driven cfgs) =====> web.xml file we configure DispatcherServlet
2. Annotation Cfgs Approach(XML + Annotation driven cfgs) =====> In userdefined class, with @Configuration set we configure "DispatcherServlet"

3. SpringBoot MVC apps

While building SpringMVC/SpringBootMVC applications we need to use the following annotations while developing handler classes

- a. @Controller => To make java class as SpringBean + handler/controller class
- b. @RequestMapping => To mark java method of @Controller class as handler method having requestpath

HTTP Methods ==> GET, POST, PUT, DELETE

Spring 4.x > To bind our method to GET Request @GetMapping

To bind our method to POST Request @PostMapping

To bind our method to Put Request @PutMapping

To bind our method to Delete Request @DeleteMapping

All this can also managed by @RequestMapping("/", method=RequestMethod.GET)//.PUT, .POST, .DELETE

Method will be binded to Get request by default if we have not specified method attribute with @RequestMapping

HandlerMapping: This component takes the incoming request through DispatcherServlet and maps the request with appropriate handler method of appropriate handler/controller class by matching the incoming url path with handler requestmethod

(Handler mapper will identify which controller will handle the request and give back Controller details to DS)

All HandlerMapping comps are the classes implementing
org.springframework.web.servlet.HandlerMapping(I)
=> Generally we work with ready made "HandlerMapping" components

ControllerClassHandlerMapping

SimpleUrlHandlerMapping

ViewResolver: This componet takes the recieve logicalViewName with physical view component returns View Object having the name and location of physical view component back to dispatcher servlet.

(view Resolver will identify extenstion and view location and this info will be sent to DS)

ViewResolver Components are the classes that implements an interface called
"org.springframework.web.servlet.ViewResolver(I)"

Generally we work with ready made "ViewResolver" components UrlBasedViewResolver

Generally we work with ready made "ViewResolver" components

UrlBasedViewResolver

InternalResourceViewResolver(default in SpringBoot)

XmlViewResolver

(AI tool for eclipse. "Codium AI")

InternalViewResolver is capable of taking jsp,html,servlet component placed in private area of view component through html,jsp components are not configured in "urlpatttern".

Outside of WEB-INF area is called public area of webapplication,the request to these components can be given directly without givingmapping details to ServletContainer.

Inside WEB-INF area is called private area of webapplication, the request to private are components must be given only after giving mapping details to ServletContainer.

(Helps to hide technology of the webapplication becoz the jsp files names does not appear in the request urls,so many things does not appear in the browser address bars.This helps to protect apps Also the source code of jsp files/components will be protected from outsider)

InternalResourceViewResolver needs 3 details to locate view components or jsp components

Location of view components --> prefix info

extension of view component--> suffix info

Logical view name(file name)

```
spring.mvc.view.prefix=/WEB-INF/views/  
spring.mvc.view.suffix=.jsp
```

In SpringBoot-MVC Apps the following components comes automatically so we need not develop them

a. DispatcherServlet

b. IOC-Container

c. HandlerMapping

d. ViewResolver

To get all these we need to go for a starter file called "spring-boot-starter-web"

=====

==>SpringBoot MVC can be developed as Standalone webapp or deployable webapplication in webserver/Application Server.

==>SpringBoot MVC Standalone web application means nothing but webapplication that uses "tomcat embeded server" so no need of arranging any external server explicitly[Good in development,testing environment]

==> This application comes as a jar file,here SpringApplication.run() itself creates tomcat embeded server along with IOC container creation and DispatcherServlet Registration, no need of sepearte ServletInitializer class.

=====

==>SpringBootMVC deployable webapplication is nothing but webapplication that is deployable in any webserver as normal war file

==>Seperate ServletInitializer class is required extending from SpringBootServlet Initializer to create IOCcontainer and register DispatcherServlet dynamically

While creating Spring-boot-mvc project if we select the type as "war", then the starter files will create 2 java files Main class cum Configuration class having @SpringBootApplication

=> The main() class is not required if we are running our

App as Deployable webapplication using External

Tomcatserver/any other server

==>ServletInitializer extending SpringBootServletInitializer

==>This class is not required if we are running our application as standalone webapplication using embeded server